RStudio, Git, and GitHub Configuration Tutorial

A step by step guide for a first time user

May Liu, Guillermo Peretz-Algorta

03-20-2025

Table of contents

0.1 Step 1: Download Git

First, you need to install Git.

To install Git, go to the Git website here: https://git-scm.com/downloads

Then, click download for for your operative system (e.g., Windows) and allow it to install. Accept the standard recommendations during the installation.

To verify Git installation, open RStudio, in the terminal, type

```
git --version
```

If you see something like git version 2.30.0, this means your Git is successfully installed.

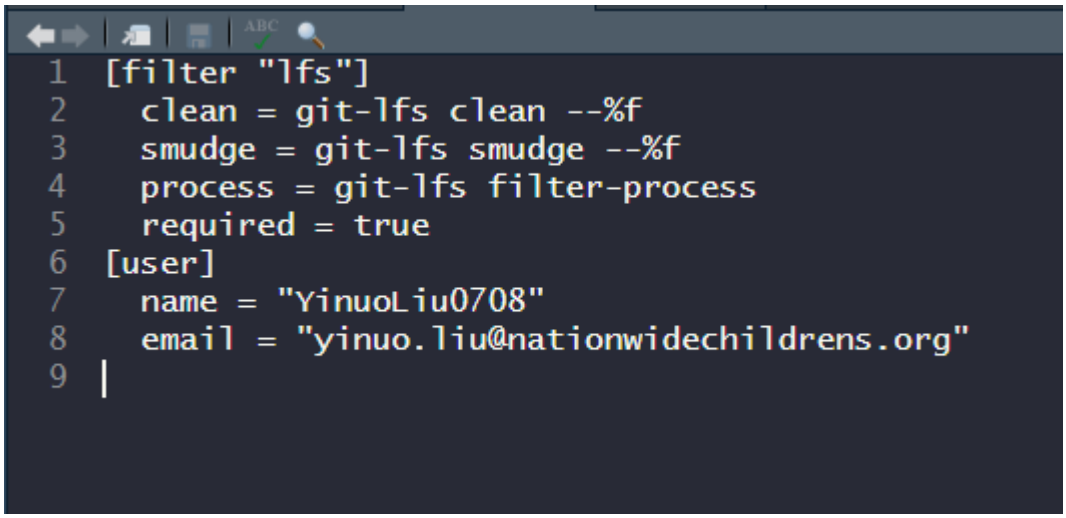
0.2 Step 2: Setting up Git in RStudio

To do the setup, we are using the R Package "usethis" Let's start running the code below in console.

```
install.packages("usethis")
library(usethis)

edit_git_config()
```

It will open a .gitconfig file in R that looks like this:



Enter your name and email address at line 7 and 8 and save the changes (Ctrl + S).

Use the same user name and email address that you used to create your Github account.

The code above [user] is to related to Git LFS (large file storage), which is an extension of Git to help manage large files like datasets, images, or videos. It's not necessary to establish git connection (only your name and email are absolutely necessary).

If you find large file storage management important for your project, feel free to copy and paste it into your .gitconfig file.

```
[filter "lfs"]
  clean = git-lfs clean -- %f
  smudge = git-lfs smudge -- %f
  process = git-lfs filter-process
  required = true
```

Be mindful of space and symbols! Incorrect space and symbols may crush the configuration. If you see error message related to .gitconfig, reopen the file with the code edit_git_config() and check if everything is correctly spelled and spaced.

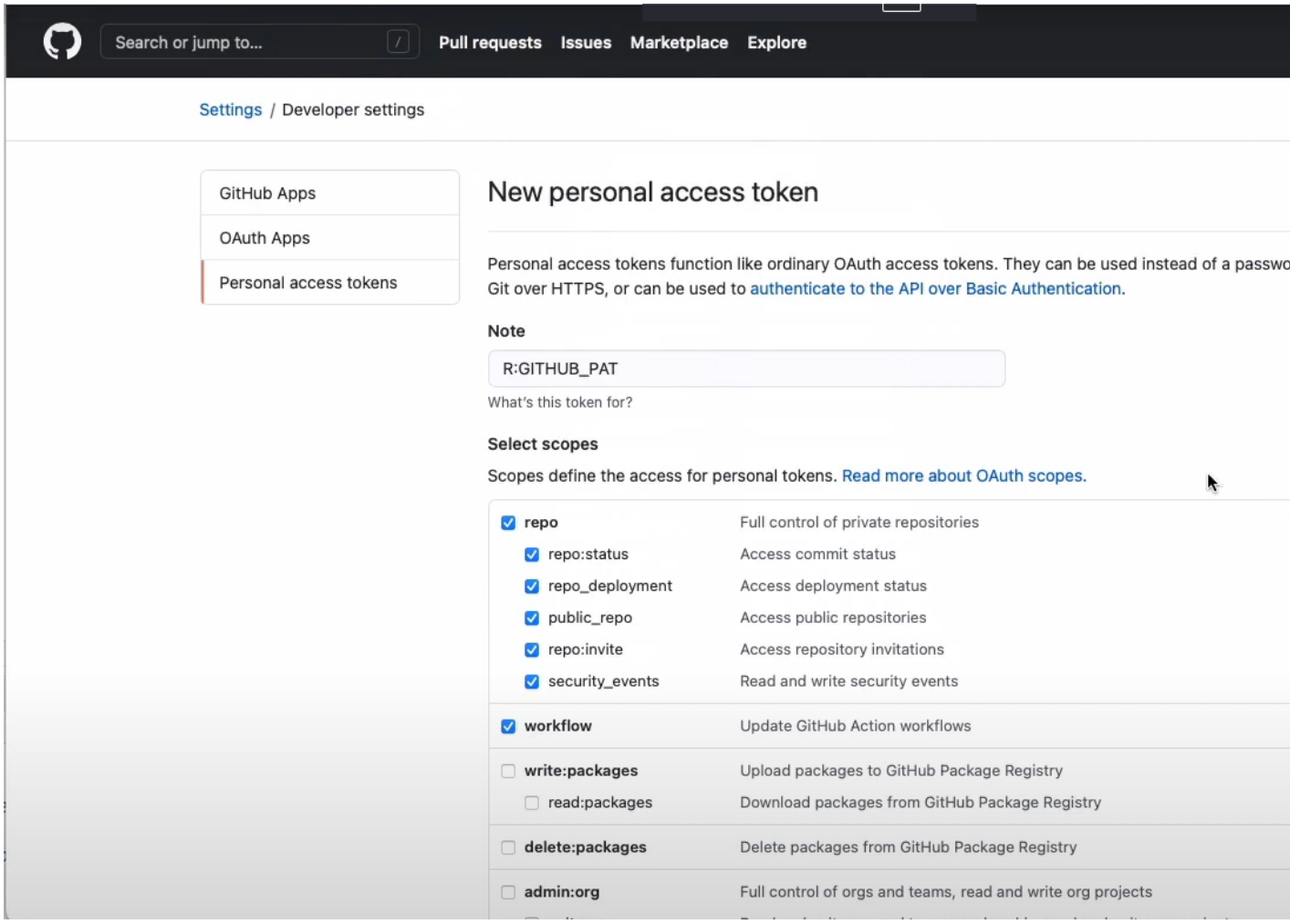
0.3 Step 3: Create a GitHub token

A personal access token in GitHub is a secure, alphanumeric string that grants access to your GitHub account and can be used instead of a password for authentication.

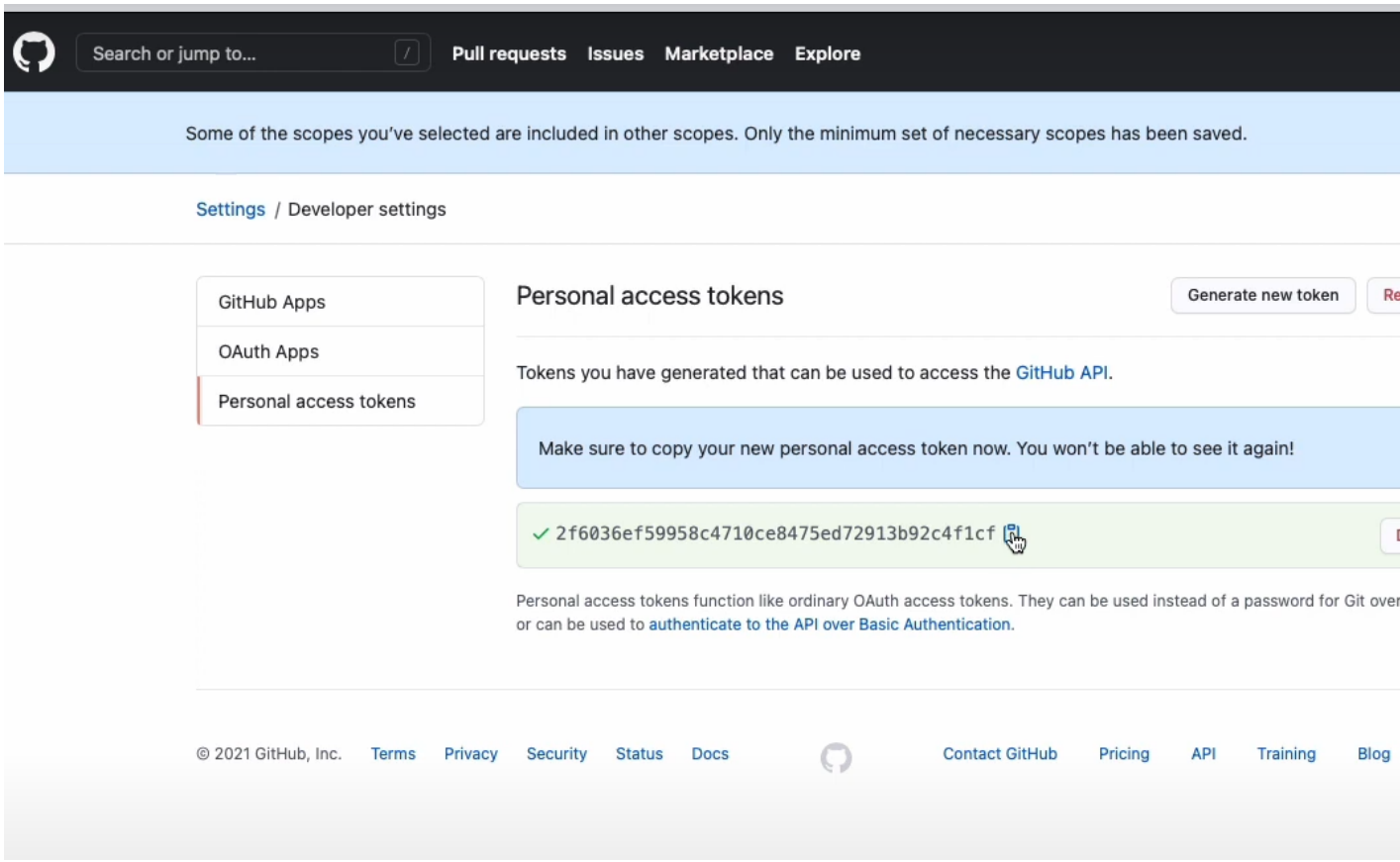
Run the following code in console to create a personal access token.

```
usethis::create_github_token()
```

It will take you to a new webpage outside of RStudio that looks like this:



Give your personal access token a name (whatever you want!). Then scroll to the bottom of the page and click 'Generate new token'. It will take you to a new page where you can copy your personal access token:



Copy your personal access token then go back to the console in Rstudio. It is recommended to save a copy of the taken in a secure file in your computer.

Typing the following code will open a dialog that asks you to enter your token.

```
gitcreds::gitcreds_set()
```

Enter you token.

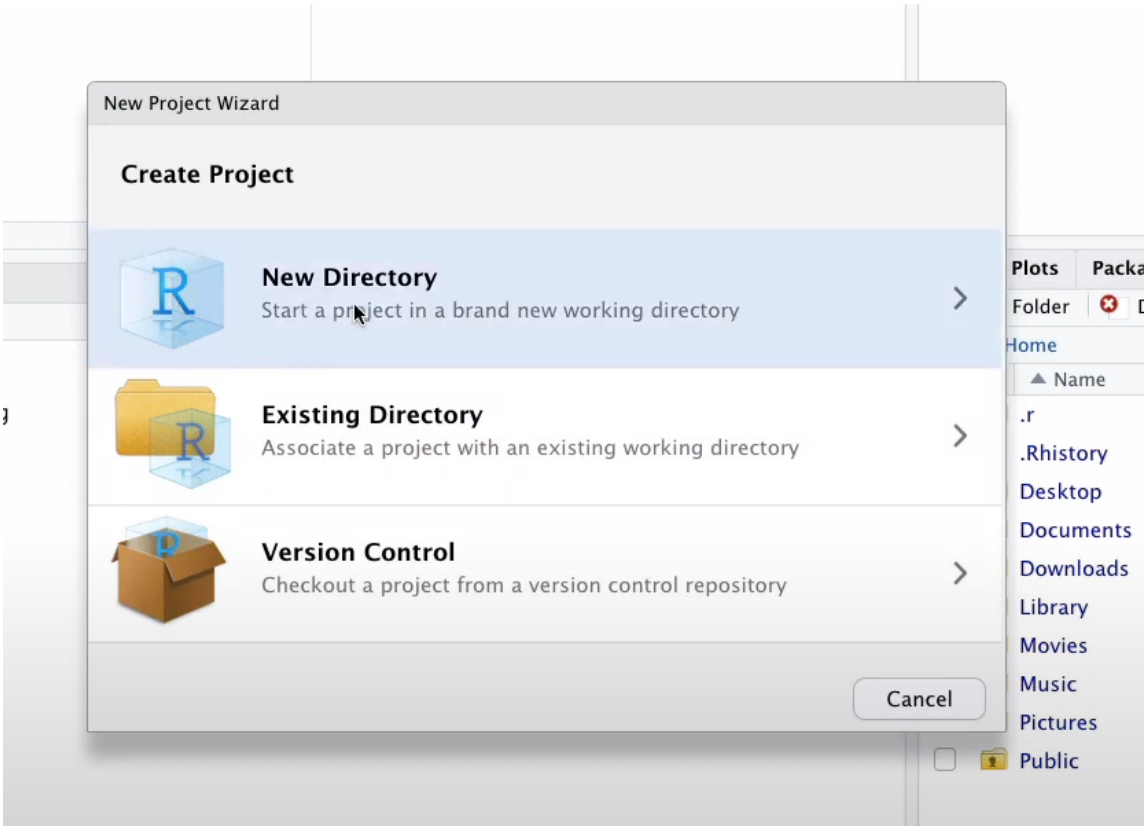
You have configured RStudio to be linked to your GitHub account.

0.4 Step 4: Create a local project and commit changes

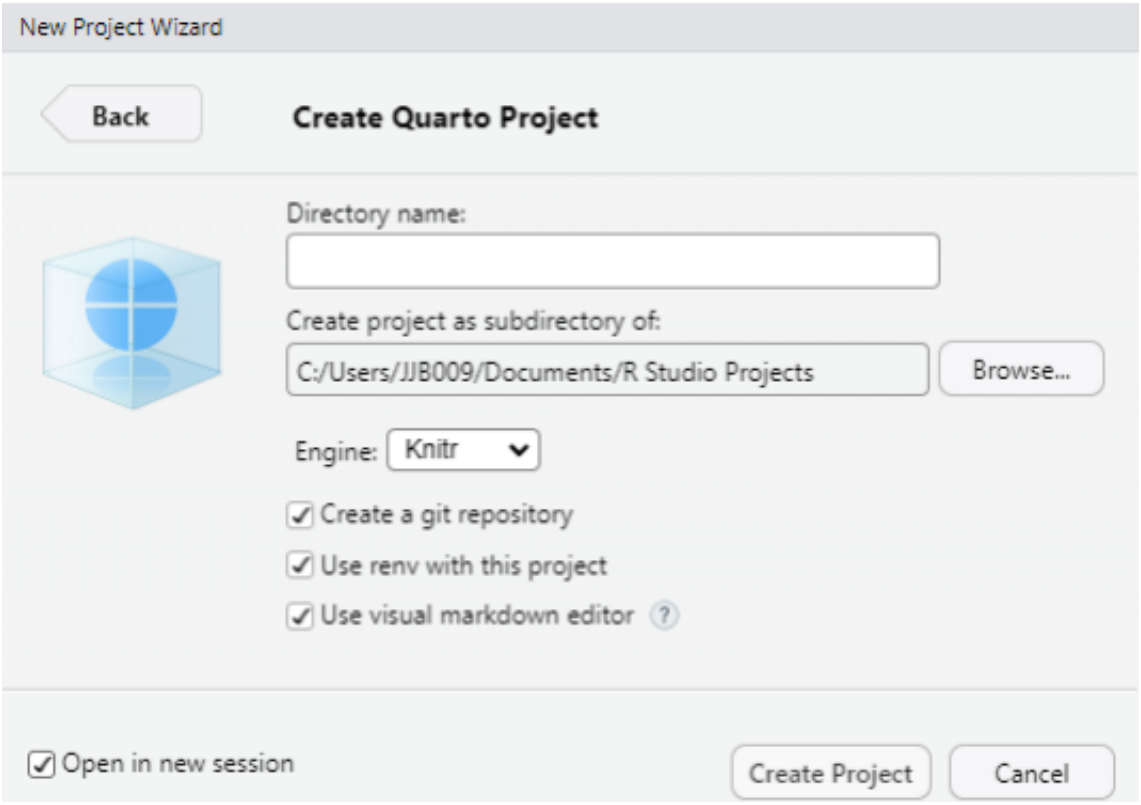
Now you can push your local project to GitHub and share it with your team.

First, you can create a new project.

In the top left corner of RStudio, go to File -> New Project, and you will see the following page



Click New Directory and choose a project type (Quarto for instance). It will take you to this page:



Give the project a name. Make sure you check all boxes. Then click Create Project. Now you are in a new project!

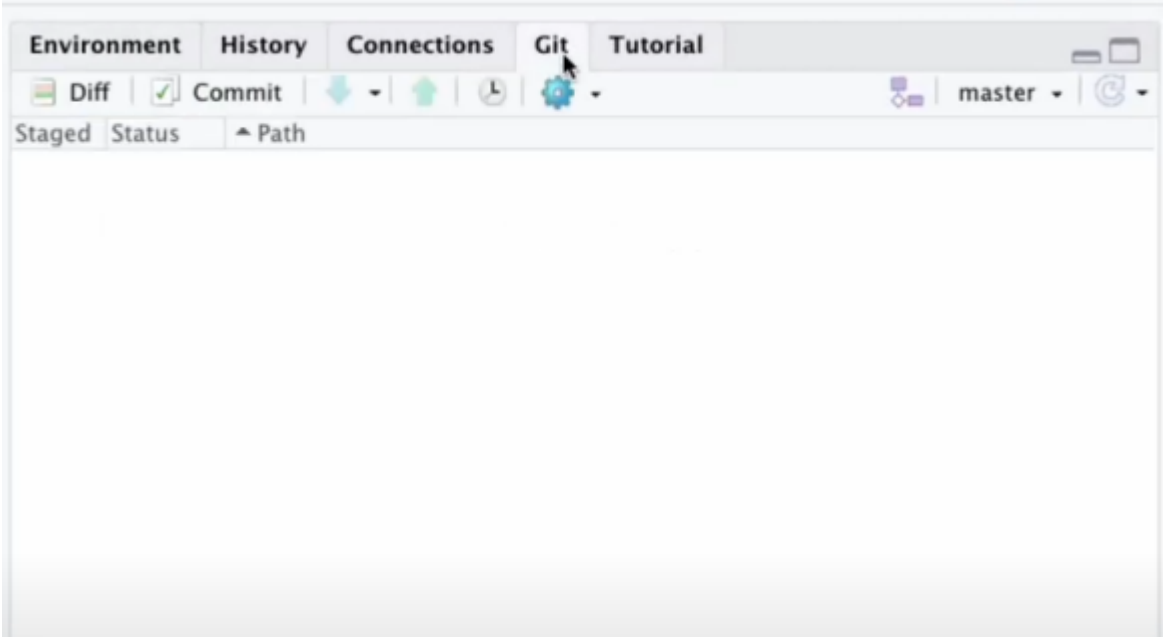
To get connected with GitHub, run the following code in the console:

```
library(usethis)
usethis::use_git()
```

When RStudio asks you questions in the console, select yes for all options (they may have it worded "definitely" or "For sure").

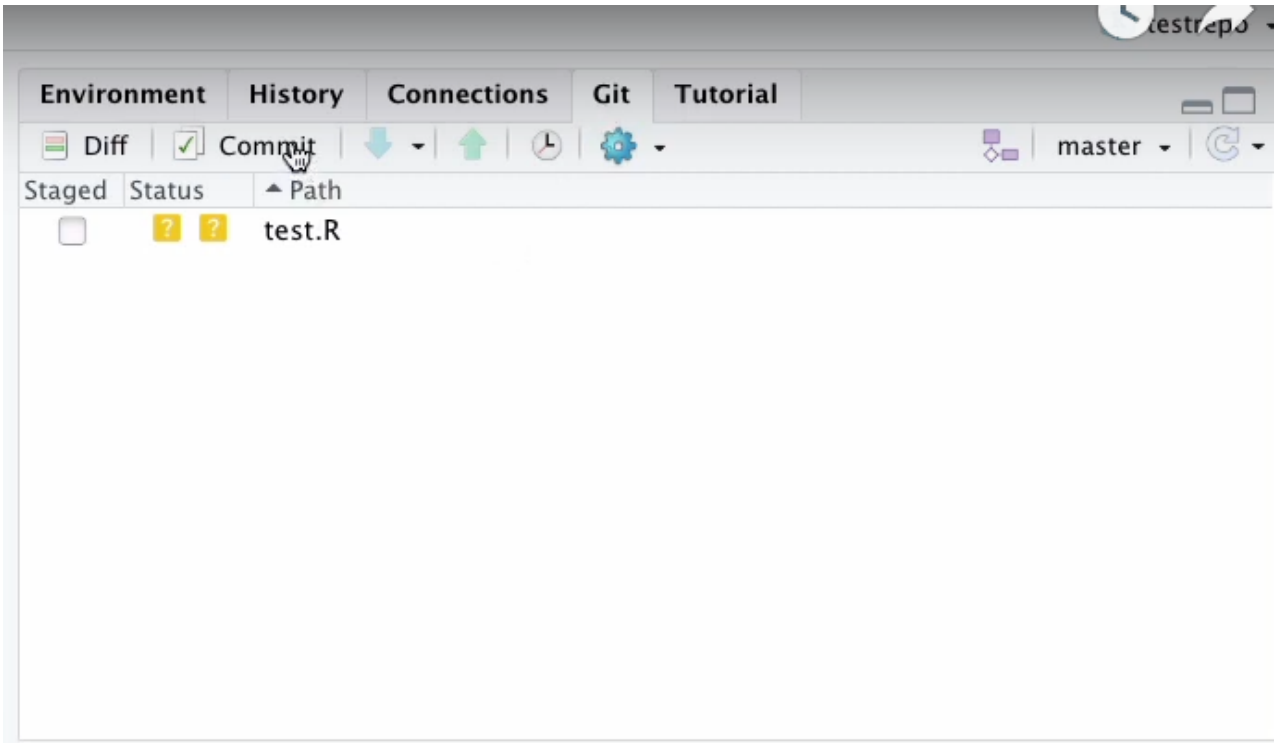
Now, you have initialized your new project as a Git repository.

Make sure that you have the Git tab in the top right section of R studio.



Create a new file in the project (could be R script, R markdown, or a random file you like). Give the file a name and save it.

You can find that there is a file pop up in the right upper corner.



You need to commit the changes you made (adding a new file in this case).

There are two ways to commit the changes: 1. type command in the terminal, 2. use the Git window

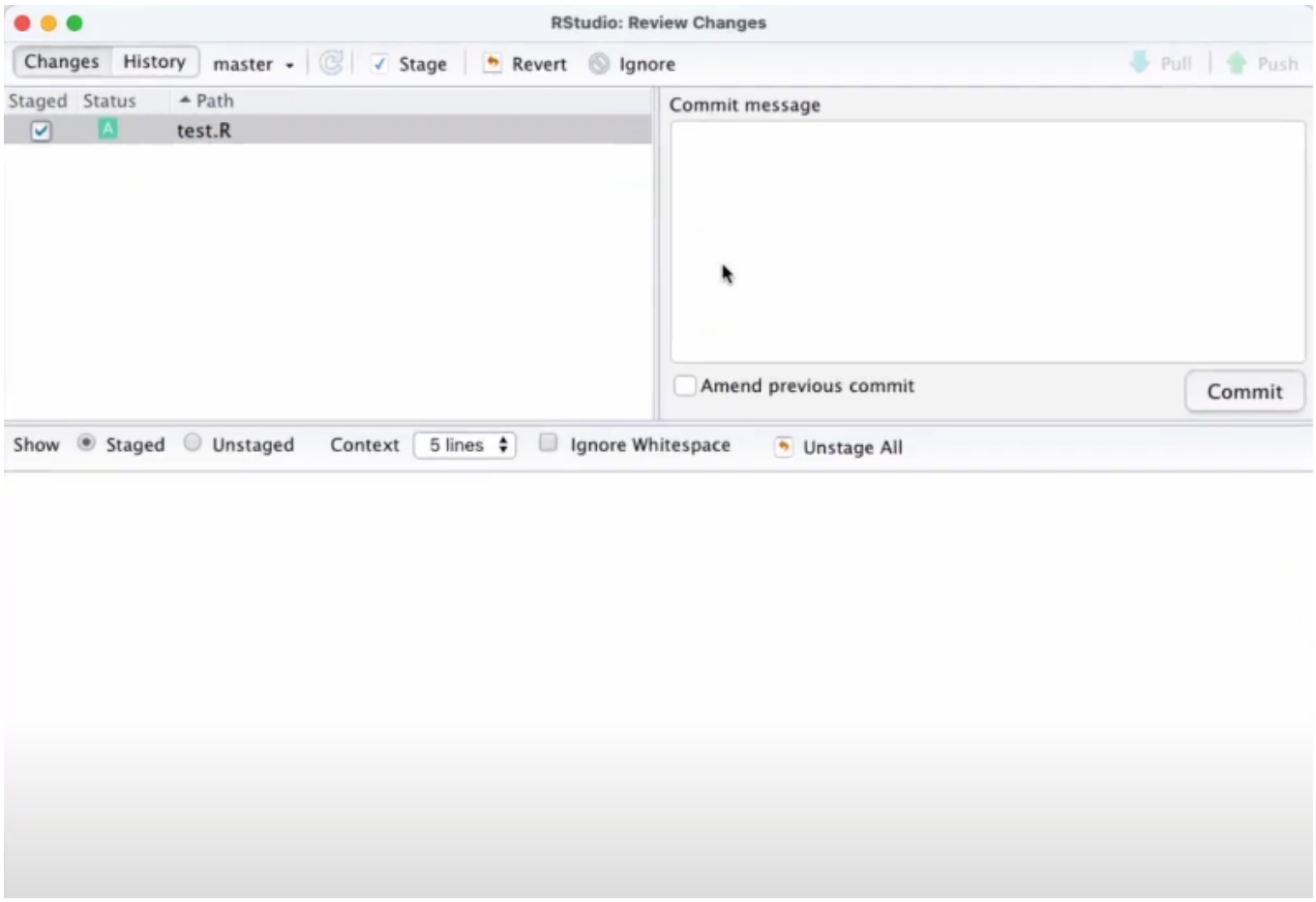
0.4.1 Commit method 1

Go to terminal, type

```
git commit -am "describe the changes you made"
```

0.4.2 Commit method 2

Click Commit in the Git window, and you will see this page



Remeber to check the staged box before the changes you want to commit.

Type in your commit message (you can't leave that box empty) and then click Commit.

We recommend directly typing commands into the terminal to make commits, as there is a variety of git commands that we will use in future team collaboration. Don't let yourself be limited in the git window in RStudio!

0.5 Step 5: Push the local project to GitHub

You have created a project and committed changes you made. The next step is to push the local project to GitHub.

In console, type the code

```
library(usethis)
use_github()
```

Now your project is pushed to GitHub!

Go to your GitHub page and find the corresponding project.

0.6 Step 6: Pull from Remote Repository

In addition to push your local files to the remote end, you can also pull stuff from the remote repository.

Type the following command in the terminal and hit Enter:

```
git pull
```

It will suggest that your files are "Already up to date."

Your local file has the same content as the remote repository, so nothing more can be pulled and incorporated into your local files.

However, when you work with multiple team members in the future, always remember to git pull before adding stuff, so you can keep up with all the changes your teammates make.

Also remember to commit changes with accurate descriptions and push your commits to the remote repository, so other members can keep track of your work as well!

0.7 Step 7: Creating Branches

New concept alert! You will learn an important feature of git – branch in the next few sections.

The branching feature allows you to create independent lines of development within a repository. This means you can work on new features, experiments, or bug fixes without affecting the main codebase.

Type the following command to see all existing branches and the branch you're at right now

```
git branch
```

There is only one master branch since you haven't created any new branches yet. The master branch is marked as green to indicate that you are here.

The master branch is the default branch in git repositories. People often treat it as the stable version of your project, and changes are merged into the master only after they've been thoroughly tested.

You may want to experiment with new code and features in your personal branch instead of the master branch (which is a good practice of coding). How to create a branch?

Here is the command:

```
git switch new-branch-name
```

Replace your preferred branch name in the command!

Try checking branches with this code

```
git branch
```

Now you will see two branches, one master, and one <whatever you call your personal branch>, and the green mark suggests that you are at the new branch you just created!

This is because the command - git switch branch-name - does two things at the same time. It creates a new branch, and then switch your working branch to the newly created one.

If you want to go back to the master branch, try

```
git switch master
```

Now you're back at the master! Great work creating new branches and switching back and forth!

0.8 Step 8: Merging Local Branches

How do we synchronize the progress on different branches? The trick is merge.

Let's first go back to the new branch you just created (command hint: git switch branch-name). Add some new content like creating a new file, writing down some random sentences. Save and commit these new changes (command hint: git commit -am "comments").

Now switch back to the master branch. You will surprisingly find that all the content you just added is not there. It's the purpose of branches to keep work separate. It's also easy to synchronize the progress on different branches with the following git command.

```
git merge name-of-branch-you-want-to-merge
```

Wait a few seconds, and you will see the newly added content shows up on your master branch.

It's important that you stay or switch to the branch that you want new changes be merged **into**, and specify the name of branch where all changes come from in the git command.

Right now, you don't need to worry about the trouble of resolving merge conflict, a condition that both branches affect the same line of code which causes conflicts. When merge conflict shows up in the future (and it will, frequently, in everyone's coding time), you will act as the judge of branches and decide how to reconcile the differences.

0.9 Step 8: Push Local Branches to Remote Repository

You want to publish your local branches in the remote repository, so team members can see them.

You can push one specific local branch to the remote end with the following command:

```
git push -u origin branch-name
```

You can also push all local branches to the remote end at once:

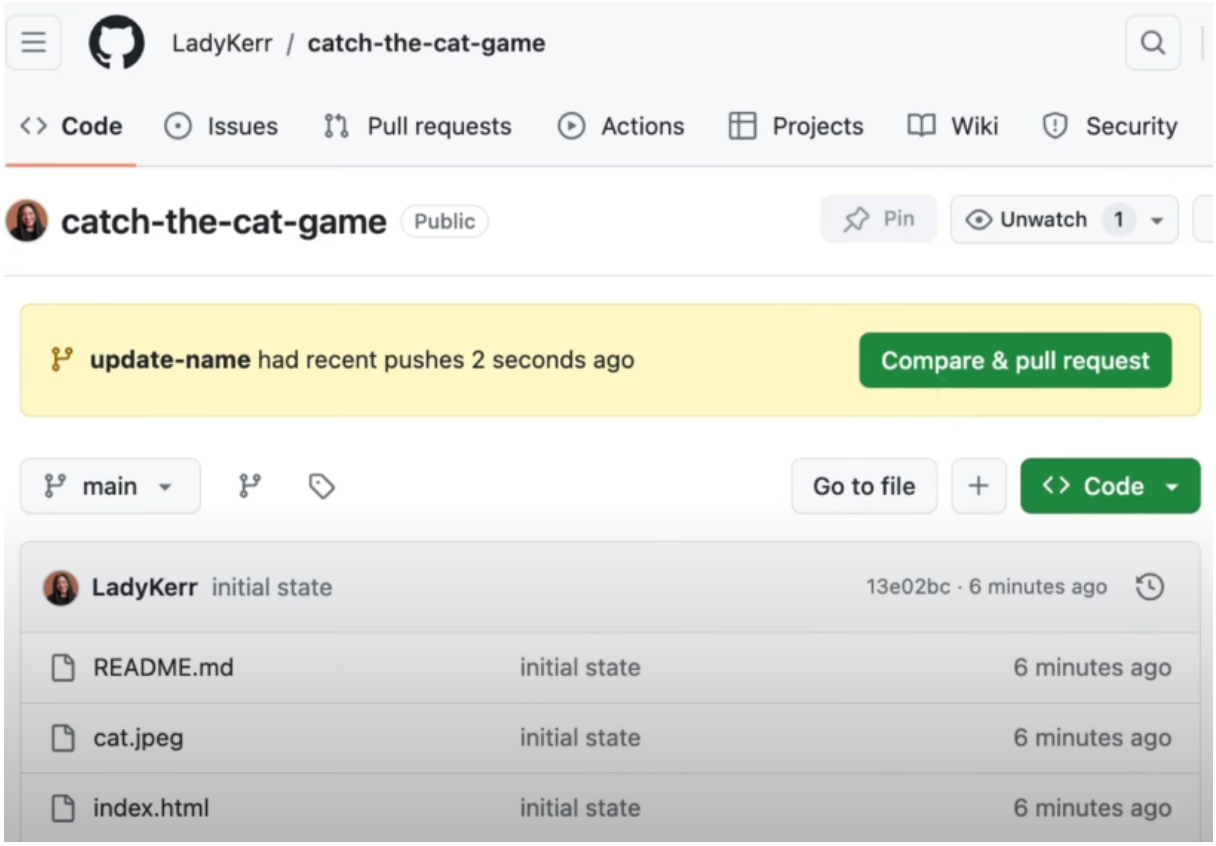
```
git push --all origin
```

It's a good practice to use different branches when working collaboratively. It promotes isolation of work, which reduces the risk of introducing bugs to the master branch. It facilitates parallel development: multiple team members can work simultaneously on different features. Though creating and interacting with branches may seem like a burden to you right now, try embracing the idea of branches and practicing it since it will make your future work easier.

0.10 Step 9: Create Pull Request on GitHub

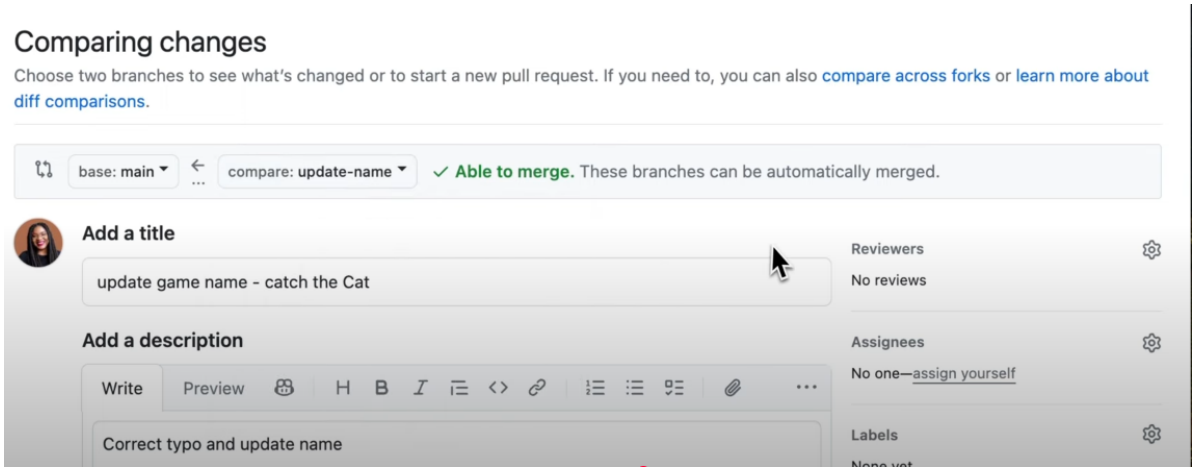
Add something new on your local branch other than the master branch, add and commit the changes (command hint: git commit -am "comments"), then push to this local branch to the remote repository (command hint: git push -u origin branch-name).

After the changes on your local branch is committed and pushed, navigate to the GitHub page. You will see a notification like this.



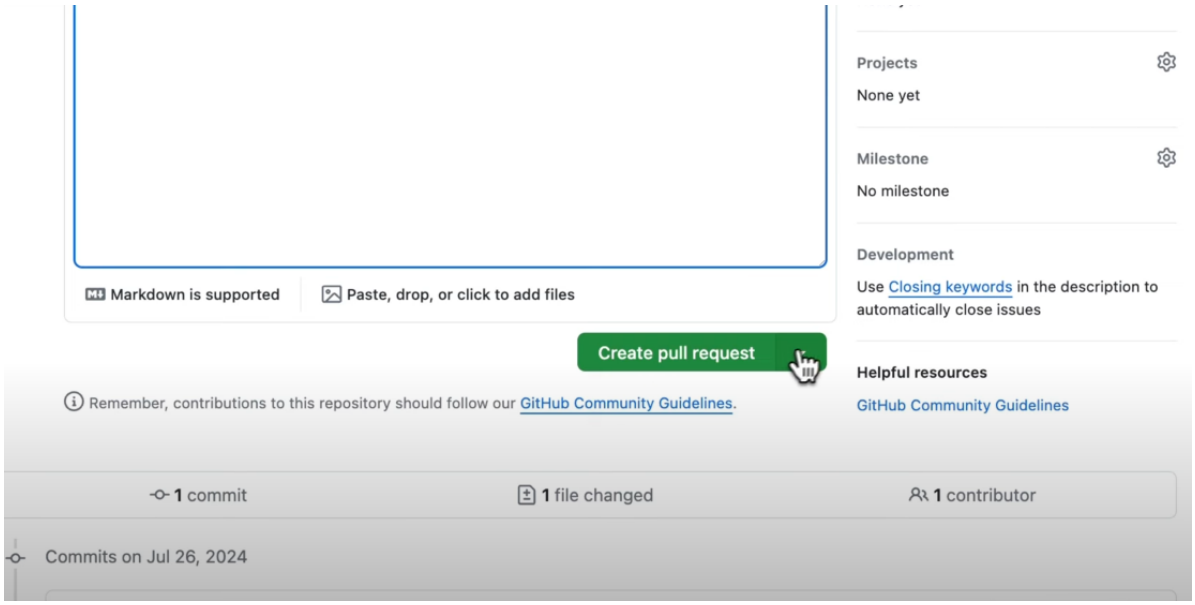
This notification occurs because your feature branch is ahead of your master branch, meaning that the feature branch has changes not in the master branch. GitHub captures this discrepancy in branches, and it wants to make sure that the master branch is up-to-date, so it suggests to make a pull request to update the master branch.

If you feel good about merging new changes to the master branch, click Compare & pull request.



You will see a page like that. Pay attention to the header. The base branch is the branch all the new changes will be merged into, and the compare branch is where the changes come from. The green words Able to merge indicates that there is no conflict between these two branches, and you can easily merge the branches.

Add a title and a description to this pull request if you want to, then scroll down and click Create pull request.



You just created your first pull request! Navigate back to the code page, check out the master branch, and you will find that the edits you made on the feature branch are there on the master branch.

If you are working with multiple team members, remember to consult other members' opinions

before creating and executing pull requests. You don't want to change the base code that everyone will work on before notifying other people.

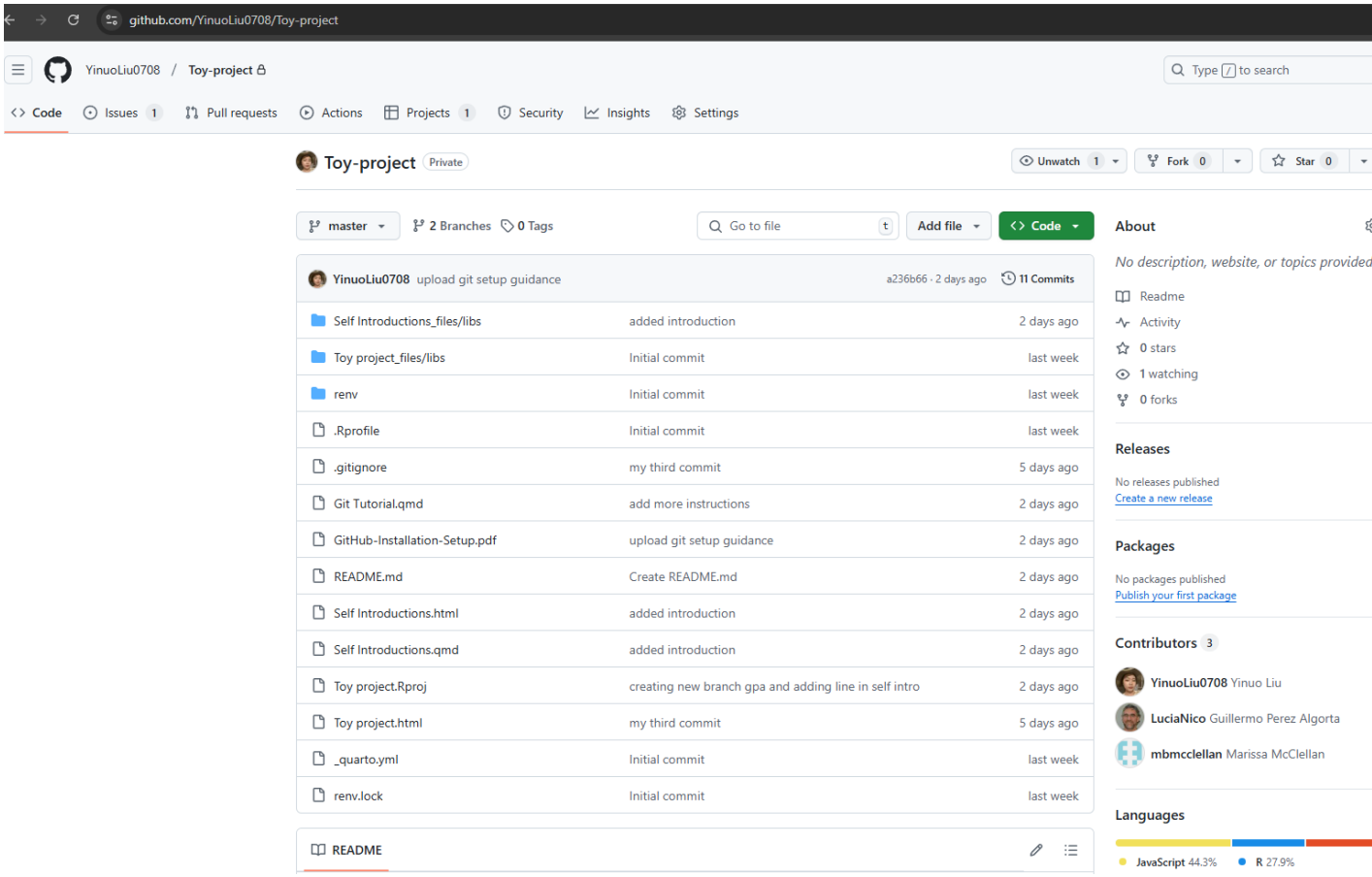
0.11 Step 10: Clone Git Repository in RStudio

Now you have some basic knowledge of Git, GitHub, and branches. It's a good time to really interact with a shared project that multiple people work on. You first need to obtain a copy of the project.

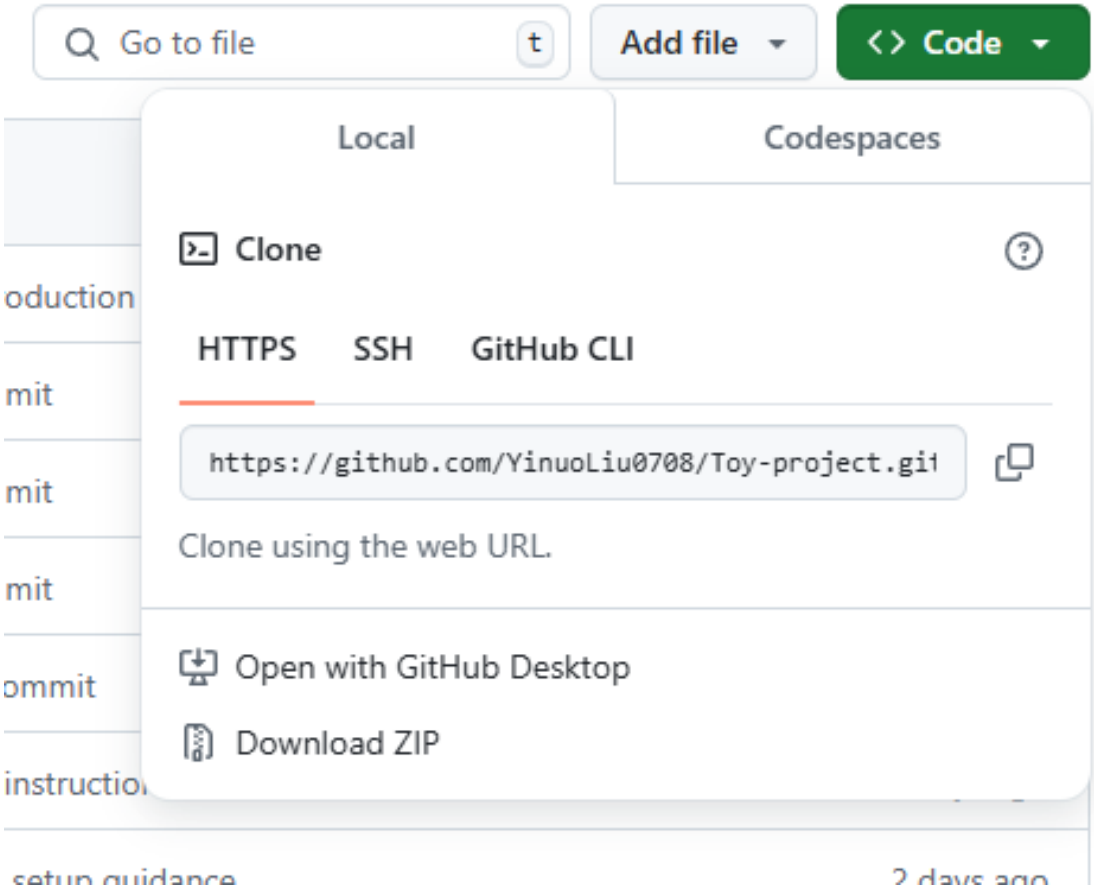
Cloning a Git repository in RStudio allows you to create a local copy of a project from GitHub.

First, you need to access the GitHub repository you want to clone.

Take this toy project for example. You will see this page

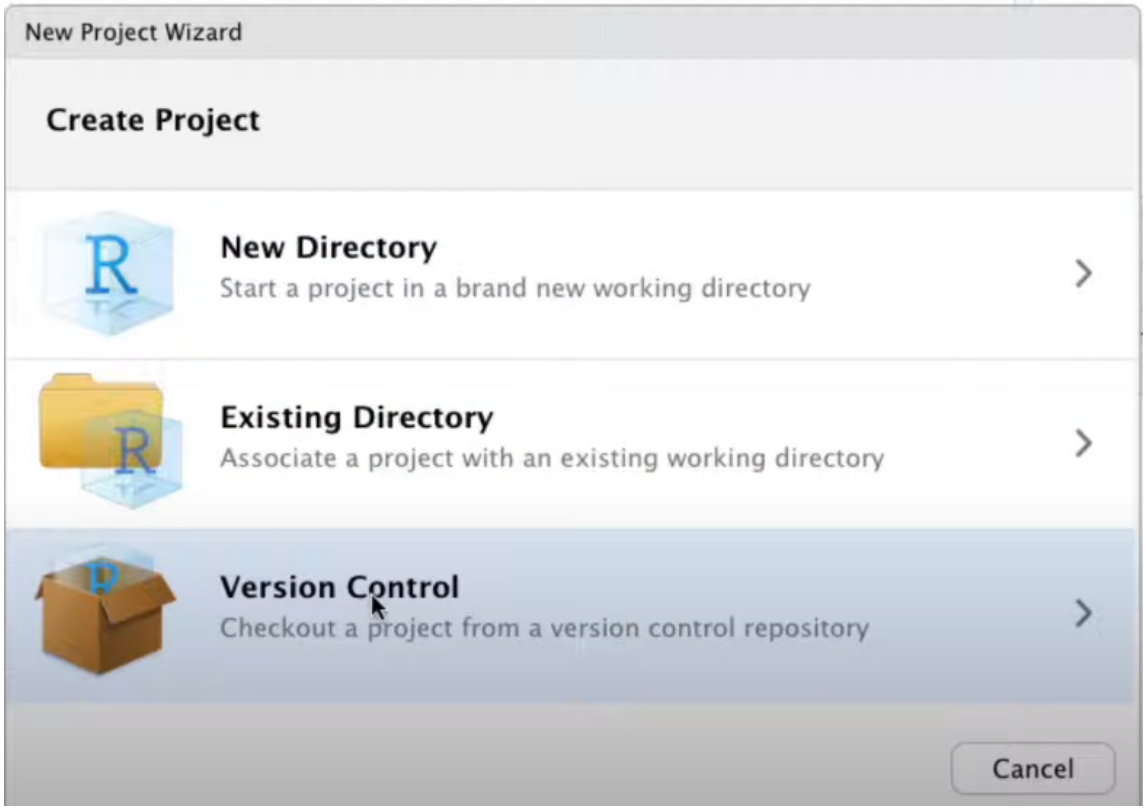


Click on the green Code button. Then, copy the http URL to the clipboard.

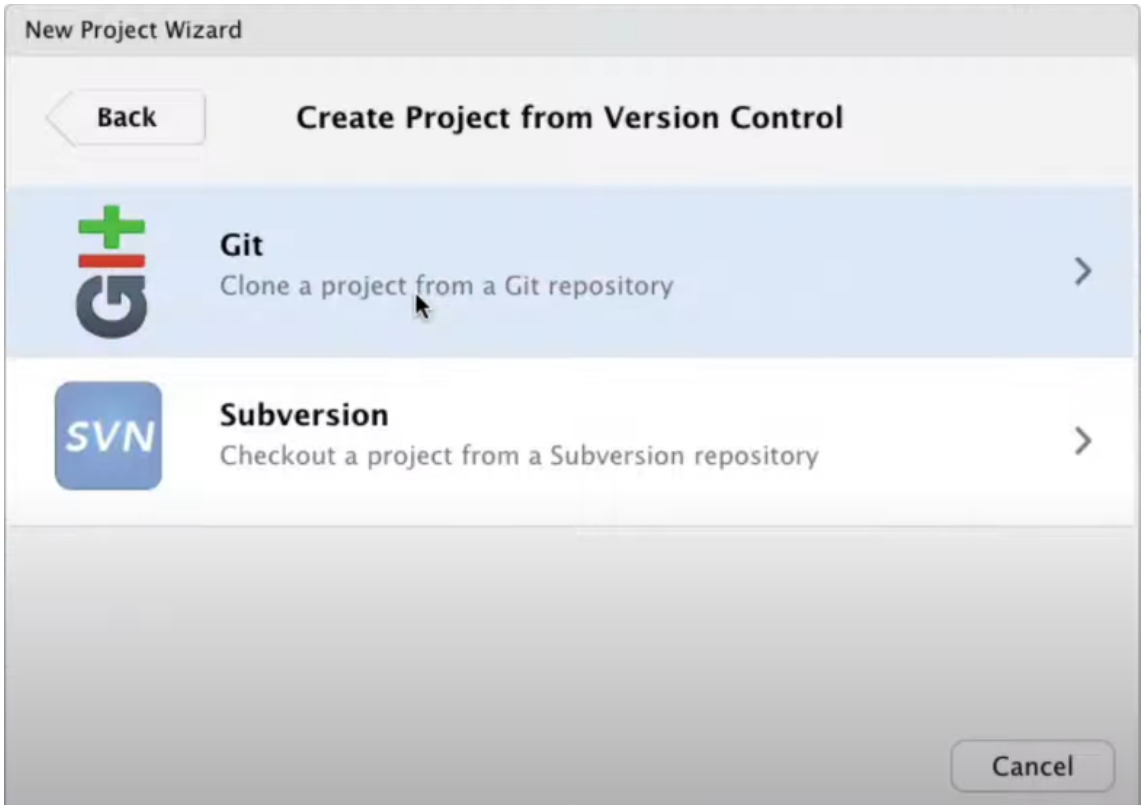


Open a new session in RStudio (Go to Session -> New Session).

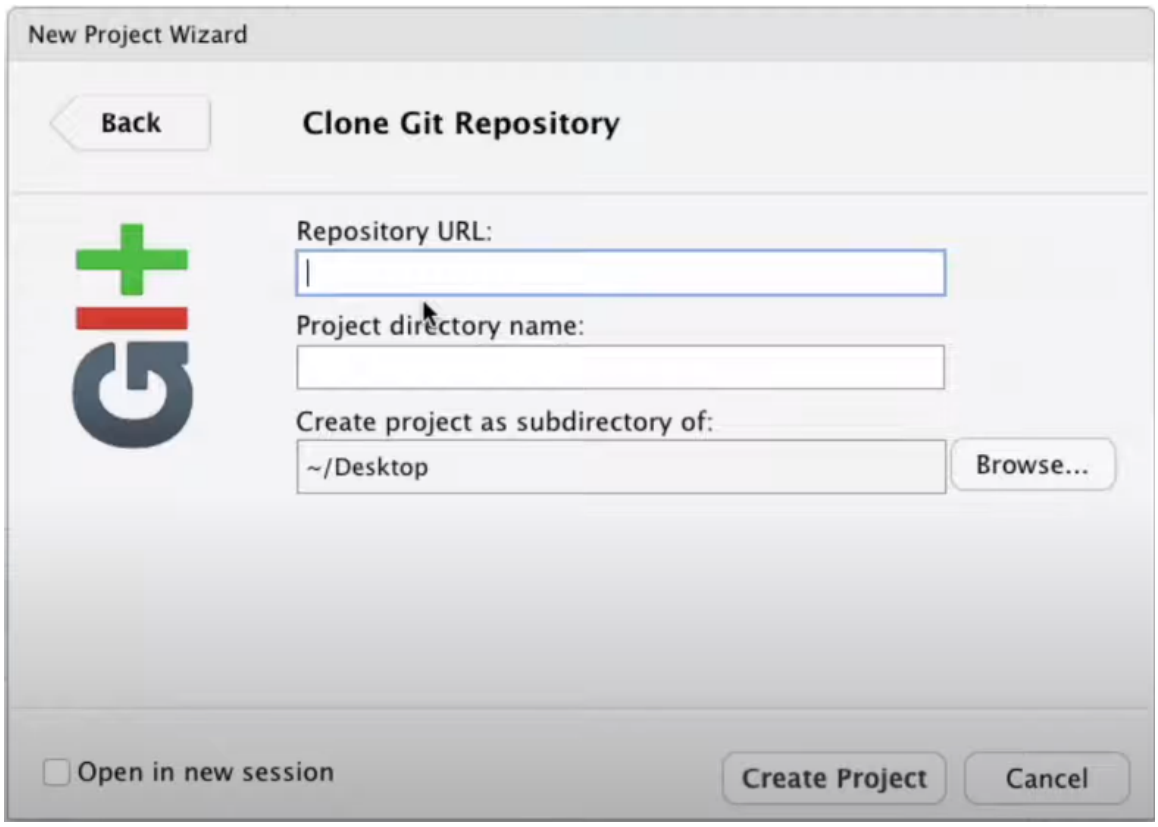
In the top left corner of RStudio, go to File -> New Project, then choose Version Control.



That will open a new window with two options, choose Git



Now, paste the link you copied from GitHub into Repository URL. Then click Create Project.



You have a copy of the Toy project from the remote depository on GitHub!

0.12 Step 11: Practice!

It's a good time to practice all the commands you learned in this tutorial.

Try the following tasks

1. create a branch named as your preferred name
2. add your introduction in Self Introductions.qmd
3. Commit all the changes you made
4. Push the local branch to the remote end